

Conclusiones sobre la lógica algorítmica

La lógica fundamental de una lista enlazada se mantuvo idéntica al cambiar de java a Python, siempre consiste en crear nodos y enlazarlos mediante referencias al nodo siguiente. La complejidad temporal también se conserva, insertar al inicio es $O(1)$ porque solo se actualiza la referencia de la cabeza, mientras que insertar al final en una lista simple sin cola es $O(n)$ porque se debe de recorrer desde la cabeza hasta el último nodo.

Esto demuestra que el algoritmo es universal porque la idea matemática no depende del lenguaje, sino del modelo lógico, como se conectan nodos y cuantos pasos se requieren para llegar a la posición.

Conclusiones sobre las implementaciones de java vs Python

Manejo de punteros:

En java se usa null para indicar que no hay una referencia, en Python el equivalente es none, conceptualmente representan lo mismo.

Sintaxis:

Python puede facilitar la implementación por su tipado dinámico y por escribir menos código. Sin embargo, exige cuidado con self, ya que en Python se debe hacer una referencia al objeto actual.

En comparación con java, Python suele sentirse más rápido de programar, pero como no obliga tanto con tipos, es importante mantener disciplina para evitar errores lógicos al manipular enlaces.

Conclusiones sobre la lista doblemente enlazada

La lista es un mejor diseño para sistemas dinámicos que necesitan movimientos bidireccionales, porque cada nodo conoce a su siguiente y a su previo. Esto permite recorrer hacia adelante y hacia atrás de forma natural, lo cual encaja perfecto en casos como undo/redo, navegación tipo back/forward o backtracking en IA.

Aunque usa un poco más de memoria por almacenar el puntero previo, se gana flexibilidad y eficiencia práctica, además al incluir un puntero cola, se logra que insertar_al_final sea $O(1)$, evitando recorrer toda la lista, lo cual mejora el rendimiento cuando se insertan muchos elementos al final.